Divide and conquer solutions

04-E01 Maximum-subarray states and interval

An interval ending at i is either A_i alone or an interval ending at i−1 extended by A_i. The latter is best when its predecessor is best, giving $end(i) = \max(A_i, end(i-1) + A_i)$, $best(i) = \max(best(i-1), end(i))$, initialized to A_1.

i	1	2	3	4	5	6	7
A[i]	5	−9	2	6	−1	4	−3
end	5	−4	2	8	7	11	8
best	5	5	5	8	8	11	11

Restart at index 3 and extend through 6: 2+6−1+4=11. The optimum is 3..6 with sum 11. All-negative nonempty input selects a largest singleton. Only the empty-permitted version may compare with zero and return no interval. Constant work per iteration gives Θ(n) time and O(1) extra space; initializing best to zero violates the nonempty contract.

04-E02 Recursion trees and fixed-constant induction

Write $n=3^L$. Depth j has 3^j nodes of size $n/3^j$, totaling n work. There are L internal levels and n leaves of cost 1, hence T(n)=n(L+1). The candidate $U(n) = n(\log_3 n + 1)$ has U(1)=1 and

$$3U(n/3) + n = n(\log_3 n - 1 + 1) + n = U(n),$$

so induction even gives equality. A guess cn yields cn+n, which cannot imply the same cn. Replacing c by c+1 does not close induction. The exact answer is $n(1+\log_3 n)=\Theta(n \log n)$. At n=9, costs 9,9,9 total 27.

04-E03 A logarithm-squared boundary

Here a=9,b=3,p=2. The ratio $f/n^p=(\lg n)^2$ is unbounded but smaller than every fixed n^ε, excluding cases 1,2,3. For $n=3^L$, depth j costs $n^2(\lg 3)^2(L-j)^2$. Put r=L−j. Then the sum is $n^2(\lg 3)^2 \sum_{r=1}^{L} r^2$, and

$$\sum_{r=1}^{L} r^2 = L(L+1)(2L+1)/6 = \Theta(L^3).$$

Leaf cost $\Theta(n^2)$ is lower order. Rounding above constant-size levels preserves these orders. The result is $\Theta(n^2 \log^3 n)$, a proved logarithmic extension rather than case 3 based only on f being larger.

Divide and conquer solutions (continued)

04-E04 Unequal splits and perturbations

Use the cited continuous theorem with fixed positive a_i,$0<b_i<1$, positive bounded bases, nonnegative differentiable g with polynomially bounded derivative, perturbations $O(x/\log^2 x)$, and child arguments in $[0,\rho x]$ for fixed $\rho<1$. The first recurrence has $a_1=a_2=1$,$b_1=1/3$,$b_2=2/3$,h=0,g=x. All conditions hold with a sufficiently large base threshold.

The continuous function $F(p)=(1/3)^p+(2/3)^p$ strictly decreases from infinity to zero, so F=1 has exactly one real root. Substitution gives p=1. Thus

$$T(x)=\Theta\left(x\left[1+\int_1^x \frac{u}{u^2}\,du\right]\right)=\Theta(x(1+\ln x)).$$

In the second recurrence the first child is 3n/4; n/4 is not $O(n/\log^2 n)$. Use ratios 3/4 and 1/2. Their unique root is $p\approx 1.507126591639$. Since

$$\int_1^x u^{-p}\,du=\frac{1-x^{1-p}}{p-1}=\Theta(1),$$

the answer is $\Theta(x^p)$. The first result is $\Theta(n \log n)$; the second exponent is defined exactly by its root equation, with the decimal only an approximation.

04-E05 Seven products and factor order

The products are M1=(2+4)(5+1)=36, M2=(3+4)5=35, M3=2(2−1)=2, M4=4(−2−5)=−28, M5=(2−1)1=1, M6=(3−2)(5+2)=7, and M7=(−1−4)(−2+1)=5. Reconstruction gives $C_{11}=36-28-1+5=12$,$C_{12}=2+1=3$,$C_{21}=35-28=7$,$C_{22}=36-35+2+7=10$.

For general blocks,

$$C_{22}=(ae+ah+de+dh)-(ce+de)+(af-ah)+(ce+cf-ae-af)=cf+dh.$$

Every canceled term has identical left/right factor order. Direct multiplication verifies C=[[12,3],[7,10]]. Scalars commute, but induction uses matrix blocks; rewriting de as ed would invalidate cancellation.

04-E06 Maximum-subarray merging without scanning

Let summaries (t,p,s,b) denote total, best nonempty prefix, suffix, and subarray. Combine adjacent L,R using

$$t=t_L+t_R,\quad p=\max(p_L,t_L+p_R),\quad s=\max(s_R,s_L+t_R),\quad b=\max(b_L,b_R,s_L+p_R).$$

Divide and conquer solutions (continued)

A prefix lies in L or contains all of L plus a prefix of R. The suffix argument reverses sides. A best interval lies left, right, or crosses. Singleton summaries all equal A_i, and endpoints follow each selected case. Induction proves all summaries, including all-negative inputs without empty intervals.

Every internal combination costs O(1). The tree has n leaves and n−1 internal nodes, so $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(1) = \Theta(n)$. CROSS returns less information and rescans at every level, so its Θ(n log n) analysis remains correct.

04-E07 Strengthening induction to pay lower-order costs

Since $5(n/2)^p = n^p$, the plain guess yields $cn^p + n^2$, which cannot be at most cn^p. The stronger guess yields $cn^p - (5d/4)n^2 + n^2$. To make this at most $cn^p - dn^2$, take $d \geq 4$. Choose d=4,c=5; the base gives $c - d = 1 = T_1$, and substitution is equality. Thus $T(n) = 5n^p - 4n^2$.

There are n^p leaves of cost 1, giving $T \geq n^p$; the exact expression gives $T \leq 5n^p$. Therefore $T = \Theta(n^{\lg 5})$. At n=2 the formula gives $25 - 16 = 9 = 5T_1 + 4$. Child-level deductions in the strengthened hypothesis pay the parent cost.

04-E08 Rounding and base cases

At n=2, ceil(10/6)=2, causing self-recursion. Sizes 3,4,5 do too. T_1 alone is insufficient. Define every size $1 \leq n \leq 6$ as a positive constant. For $n \geq 7$, $\lceil 5n/6 \rceil \leq n - 1$, ensuring termination.

Put b=6/5 and $q_{j+1} = \lceil q_j / b \rceil$. If $|q_j - n/b^j| \leq C$, next error is at most C/b+1; C=6 closes induction. To avoid the upper error bound landing exactly on the base boundary, bases may instead cover $n \leq 12$. Then n/b^j entering a fixed interval implies termination within $\log_b n + O(1)$ levels. Conversely $q_j \geq n/b^j$ supplies $\Omega(\log n)$ levels. Hence repaired time is Θ(log n). Either base interval 6 or 12 works; only the latter makes this bounded-error upper proof immediate.

04-E09 Regularity cannot be omitted

For every n, $f \geq n^2$, giving polynomial separation with p=1,ε=1. For even L, n/2 has odd exponent, so $T(n) \geq 2f(n/2) = n^3/4$. Current $f = n^2$, hence $T/f \geq n/4 \to \infty$. Any eventual $T \leq Cf$ would require $n/4 \leq C$ on this unbounded subsequence, impossible.

Also $2f(n/2)/f(n) = n/4$, so no fixed $c < 1$ gives regularity on every sufficiently large input. Positive bases and nonnegative costs cannot replace regularity. T is not Θ(f); this violates no theorem because a hypothesis fails.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Divide and conquer solutions (continued)

04-E10 Finer boundaries than polynomial separation

Let $n=2^L$ and stop at a fixed $L_0 \geq 1$. With p=1, depth j costs respectively $n/(L-j)$ and $n/(L-j)^2$. Setting r=L−j gives $n \sum r^{-1}$ and $n \sum r^{-2}$, with r ranging from a fixed positive integer to L. The first sum is Θ(log L); the second Θ(1). Both also have Θ(n) leaf costs. Restore L=lg n to obtain Θ(n log log n) and Θ(n).

Neither is classical case 1: logarithmic denominators do not give an n^ε gap for any fixed ε>0.

04-E11 Parameter copying changes time

Views compare the midpoint and pass indices, giving $V(n) = V(\lfloor n/2 \rfloor) + \Theta(1)$, hence worst-case Θ(log N). Current-interval copying gives $C(n) = C(\lfloor n/2 \rfloor) + \Theta(n)$. Its path costs N+N/2+…=Θ(N). Copying the original array gives $F(n; N) = F(\lfloor n/2 \rfloor; N) + \Theta(N)$, with Θ(log N) worst-case levels and Θ(N log N) time.

Original N stays fixed while current n shrinks; replacing Θ(N) by Θ(n) changes the model. Failed search or deepest targets establish matching lower depths. A root hit instead costs respectively Θ(1),Θ(N),Θ(N), so the stated results are worst-case bounds.

04-E12 Restore the original variable

Let $m=\lg n=2^k$,$S(m)=T(2^m)$. Then $S_1=T_2=1$ and $S(m)=3S(m/2)+m$. Its critical exponent $p=\lg 3$ exceeds 1; the linear toll is polynomially smaller, giving $S=\Theta(m^p)$. Alternatively depth j costs $m(3/2)^j$ and leaves cost $3^k=m^p$; the geometric sum has this same order.

Restoring variables gives $T(n) = \Theta((\lg n)^{\lg 3})$ on the specified domain $n = 2^{2^k}$. The proposed Θ(n^(lg 3)) mistakes new m for original n. Do not rename the variable back before restoration.

Divide and conquer solutions (continued)

Verification code appendix: akra_bazzi.py

```
"""Check the arithmetic in the Akra-Bazzi supplement."""

from math import log


TERMS = ((1.0, 1.0 / 3.0), (1.0, 2.0 / 3.0))


def characteristic(p, terms=TERMS):
    return sum(coefficient * fraction**p for coefficient, fraction in terms)


def solve_exponent(terms=TERMS):
    lo, hi = -20.0, 20.0
    for _ in range(200):
        mid = (lo + hi) / 2.0
        if characteristic(mid, terms) > 1.0:
            lo = mid
        else:
            hi = mid
    return (lo + hi) / 2.0


def integral_for_g_power(n, p, exponent):
    if abs(exponent - p) < 1e-12:
        return n**p * log(n)
    return (n**exponent - n**p) / (exponent - p)

def master_case_one_bound(a, b, epsilon, n_power):
    p = log(a) / log(b)
    ratio = a / b ** (p - epsilon)
    depth = int(log(n_power, b))
    level_sum = sum(
        n_power ** (p - epsilon) * ratio**level
        for level in range(depth + 1)
    )
    return p, ratio, depth, level_sum
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Divide and conquer solutions (continued)

```
def main():
    balanced_unequal = solve_exponent()
    assert abs(balanced_unequal - 1.0) < 1e-12
    assert abs(integral_for_g_power(64, 1.0, 1.0) - 64 * log(64)) < 1e-10

    # Keeping b=1/2 for the first call is invalid after a perturb of n/4.
    naive = ((1.0, 0.5), (1.0, 0.5))
    corrected = ((1.0, 0.75), (1.0, 0.5))
    naive_p = solve_exponent(naive)
    corrected_p = solve_exponent(corrected)
    assert abs(naive_p - 1.0) < 1e-12
    assert corrected_p > 1.4
    case_one_p, case_one_ratio, case_one_depth, case_one_sum = master_case_one_bound(
        2.0, 2.0, 0.5, 1024
    )
    assert case_one_p == 1.0
    assert case_one_ratio > 1.0
    assert abs(case_one_ratio**case_one_depth - 1024**0.5) < 1e-10
    assert case_one_sum <= 4 * 1024
    zero_t, zero_g, invalid_positive_lower = 0, 0, 1024
    assert zero_t == zero_g == 0 and invalid_positive_lower > zero_t
    print(
        "PASS "
        f"unequal_p={balanced_unequal:.12f} "
        f"integral_64={integral_for_g_power(64, 1.0, 1.0):.6f} "
        f"invalid_h_naive_p={naive_p:.12f} "
        f"corrected_base_p={corrected_p:.12f} "
        f"master_case1_ratio={case_one_ratio:.12f} "
        f"master_case1_sum={case_one_sum:.6f} "
        "zero_counterexample=T=0,g=0"
    )


if __name__ == "__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Divide and conquer solutions (continued)

Verification code appendix: divide_conquer_checks.py

```
"""Independent finite checks of Chapter 4 calculations; not proofs of asymptotic bounds."""
from itertools import product
from math import log,ceil
from random import Random
from collections import Counter

SAMPLE=[-2,1,-3,4,-1,2,1,-5,4]

def crossing(a,lo,mid,hi):
    total=0;left=float('-inf');li=mid
    for i in range(mid,lo-1,-1):
        total+=a[i]
        if total>left:left=total;li=i
    total=0;right=float('-inf');rj=mid+1
    for j in range(mid+1,hi):
        total+=a[j]
        if total>right:right=total;rj=j
    return li,rj+1,left+right

def divide(a,lo=0,hi=None):
    hi=len(a) if hi is None else hi
    if hi-lo==1:return lo,hi,a[lo]
    mid=(lo+hi-1)//2
    return max((divide(a,lo,mid+1),divide(a,mid+1,hi),crossing(a,lo,mid,hi)),key=lambda
t:t[2])

def summary(a):
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Divide and conquer solutions (continued)

```
    if len(a)==1:return (a[0],)*4
    mid=len(a)//2;t1,p1,s1,b1=summary(a[:mid]);t2,p2,s2,b2=summary(a[mid:])
    return t1+t2,max(p1,t1+p2),max(s2,s1+t2),max(b1,b2,s1+p2)

def naive(a,b):return [[sum(x*y for x,y in zip(row,col)) for col in zip(*b)] for row in a]
def add(a,b,sign=1):return [[x+sign*y for x,y in zip(r,s)] for r,s in zip(a,b)]
def split(a):
    m=len(a)//2
    return [r[:m] for r in a[:m]],[r[m:] for r in a[:m]],[r[:m] for r in a[m:]],[r[m:] for r
in a[m:]]
def strassen(A,B):
    if len(A)==1:return [[A[0][0]*B[0][0]]]
    a,b,c,d=split(A);e,f,g,h=split(B)
    M1=strassen(add(a,d),add(e,h));M2=strassen(add(c,d),e)
    M3=strassen(a,add(f,h,-1));M4=strassen(d,add(g,e,-1))

M5=strassen(add(a,b),h);M6=strassen(add(c,a,-1),add(e,f));M7=strassen(add(b,d,-1),add(g,h))
    c11=add(add(add(M1,M4),M5,-1),M7);c12=add(M3,M5)
    c21=add(M2,M4);c22=add(add(add(M1,M2,-1),M3),M6)
    return [r+s for r,s in zip(c11,c12)]+[r+s for r,s in zip(c21,c22)]

def check():
    arrays=0
    for n in range(1,8):
        for t in product(range(-2,3),repeat=n):
            brute=max(sum(t[i:j]) for i in range(n) for j in range(i+1,n+1))
            lo,hi,v=divide(t)
            assert v==brute==sum(t[lo:hi])==summary(t)[3];arrays+=1
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Divide and conquer solutions (continued)

```
    assert crossing(SAMPLE,0,4,9)==(3,7,6)
    rng=Random(427);matrices=0
    for n in [1,2,4,8]:
        for _ in range(50):
            A=[[rng.randrange(-6,7) for _ in range(n)] for _ in range(n)]
            B=[[rng.randrange(-6,7) for _ in range(n)] for _ in range(n)]
            assert strassen(A,B)==naive(A,B);matrices+=1
    assert strassen([[2,-1],[3,4]],[[5,2],[-2,1]])==[[12,3],[7,10]]
    # Free noncommutative polynomials: distinguish a*e from e*a.
    sym=lambda s:{s:1}
    def plus(*terms):
        out=Counter()
        for sign,v in terms:
            for key,c in v.items():out[key]+=sign*c
        return {k:v for k,v in out.items() if v}
    def mul(a,b):
        out=Counter()
        for x,c in a.items():
            for y,d in b.items():out[x+y]+=c*d
        return dict(out)
    a,b,c,d,e,f,g,h=map(sym,'abcdefgh')
    M1=mul(plus((1,a),(1,d)),plus((1,e),(1,h)))
    M2=mul(plus((1,c),(1,d)),e);M3=mul(a,plus((1,f),(-1,h)))
    M4=mul(d,plus((1,g),(-1,e)));M5=mul(plus((1,a),(1,b)),h)
    M6=mul(plus((1,c),(-1,a)),plus((1,e),(1,f)))
    M7=mul(plus((1,b),(-1,d)),plus((1,g),(1,h)))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Divide and conquer solutions (continued)

```
    assert plus((1,M1),(1,M4),(-1,M5),(1,M7))=={'ae':1,'bg':1}
    assert plus((1,M3),(1,M5))=={'af':1,'bh':1}
    assert plus((1,M2),(1,M4))=={'ce':1,'dg':1}
    assert plus((1,M1),(-1,M2),(1,M3),(1,M6))=={'cf':1,'dh':1}
    for L in range(12):
        n=2**L
        if L==0:t=1
        else:t=5*t+n*n
        assert t==5*5**L-4*n*n
    rounding=0
    from fractions import Fraction
    for b in [Fraction(6,5),Fraction(3,2),Fraction(2),Fraction(7,3)]:
        C=b/(b-1)
        for n in range(1,501):
            for use_ceil in [False,True]:
                q=n
                for j in range(20):
                    assert abs(q-Fraction(n,b**j))<=C
                    x=Fraction(q,b);q=ceil(x) if use_ceil else
x.numerator//x.denominator;rounding+=1
    return
{'array_cases':arrays,'recursive_matrix_pairs':matrices,'noncommutative_output_identities':4,
'rounding_states':rounding,'sample_crossing_one_based':[4,7,6],'strassen_sample':[[12,3],
[7,10]],'scope':'Finite examples and symbolic identities only; induction and complexity
proofs reside in the chapter.'}

if __name__=='__main__':
    import json
    print(json.dumps(check(),ensure_ascii=False,indent=2))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Divide and conquer solutions (continued)

Verification code appendix: maximum_subarray.py

```
"""Compare divide-and-conquer and linear maximum-subarray algorithms."""

def crossing(a, low, mid, high):
    left_sum = float("-inf")
    total = 0
    for i in range(mid, low - 1, -1):
        total += a[i]
        left_sum = max(left_sum, total)
    right_sum = float("-inf")
    total = 0
    for j in range(mid + 1, high + 1):
        total += a[j]
        right_sum = max(right_sum, total)
    return left_sum + right_sum


def divide(a, low=0, high=None):
    if high is None:
        high = len(a) - 1
    if low == high:
        return a[low]
    mid = (low + high) // 2
    return max(divide(a, low, mid), divide(a, mid + 1, high), crossing(a, low, mid, high))


def linear(a):
    best_end = best = a[0]
    for value in a[1:]:
        best_end = max(value, best_end + value)
        best = max(best, best_end)
    return best


sample = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
assert divide(sample) == linear(sample) == 6
print(divide(sample))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study